AI-Assisted Conversion of R Statistical Packages to Python: A Methodological Framework

Yufei Cai

Jun Li

Department of Applied and Computational Mathematics and Statistics

University of Notre Dame

{ycai9, jun.li}@nd.edu

Submitted for review

2026

Abstract

The R statistical computing environment hosts a large body of validated, high-performance, typical statistical package implementations that are unavailable as native Python libraries. Converting these packages manually is time-consuming, error-prone, and fails to scale, while runtime bridging solutions such as rpy2 [1] and reticulate [2] require an R installation and introduce inter-process overhead. Large language models (LLMs) offer a path towards automation [3, 4, 5], but unguided translation of statistical code introduces silent numerical errors arising from semantic differences between R and Python that are non-obvious from source inspection alone. Here we present a seven-phase AI-assisted methodology for the systematic, reproducible conversion of R packages to Python, implemented using a structured hierarchy of orchestrating skills (top-level slash commands) and specialized sub-agents within the Claude Code AI development environment. The methodology systematically addresses each source of translation error: it catalogues every language-specific call site and generates dedicated machine-readable translation guides before any code is written; it converts functions in topological dependency order; and it validates output numerically against the live R implementation at a specified tolerance. We demonstrate the framework on `KernSmooth` [6] (v. 2.23-26), a recommended R package implementing kernel smoothing methods [7, 8, 9]. The resulting Python package passes 518 assertion tests against the R reference under Python 3.14, achieving agreement to 6–10 significant figures across all seven public functions.

1 Introduction

The R language and the Python scientific computing stack are the two dominant platforms for quantitative research [10, 11, 12]. Despite this complementarity, the two ecosystems largely remain siloed: R's CRAN repository harbours thousands of validated statistical implementations—including *recommended* packages bundled with every R installation—that have no native Python equivalent. Bridging libraries such as `rpy2` [1] and `reticulate` [2] can call R functions from Python at runtime, but this approach requires a full R installation as a runtime dependency, incurs inter-process communication overhead, and limits deployment to environments where both runtimes coexist. For applications that demand portability, containerisation, or embedding in Python-first infrastructure, a self-contained Python port—one that requires neither an R runtime nor any external bridging layer—is strongly preferable.

Manual porting is the traditional remedy, but it does not scale. It requires expert knowledge of both languages' numerical semantics, is inconsistently documented across projects, and is prone to silent errors. Language-specific conventions that are invisible from source inspection can produce numerically incorrect output without raising any exception: R's `fft(inverse=TRUE)` returns an unnormalised inverse DFT while NumPy's `np.fft.ifft` normalises by $1/N$ by convention; R's `var(x)` applies Bessel's correction while `np.var` does not by default; Fortran `INTEGER` is 32 bits while NumPy defaults to 64 bits; and R evaluates default arguments lazily at first use while Python evaluates them eagerly at call time. Any of these discrepancies, if unaddressed, can corrupt numerical results without raising an error.

Large language models have recently demonstrated substantial capability for cross-language code translation [3, 4, 5]. Unsupervised neural machine translation between programming languages can achieve competitive functional equivalence rates [4], and LLM-based approaches with formal verification can provide correctness guarantees on translated code [13]. Scalable, validated translation of entire software projects using LLMs has been demonstrated on general-purpose codebases [14]. However, statistical scientific software presents challenges that these approaches do not directly address: the correctness criterion is numerical equivalence of floating-point output rather than structural or logical equivalence; the relevant domain knowledge (statistical algorithms, Fortran FFI conventions, R numeric semantics) is diffuse and underrepresented in LLM training corpora; and silent numerical errors are far more consequential than runtime crashes for scientific reproducibility.

A further challenge is consistency. Each converted function must use identical idioms for recurring patterns—FFT normalisation, integer coercions, missing-argument detection—regardless of the order in which conversion is performed. An agent operating on a single function without awareness of decisions made for other functions will produce code that is individually plausible but collectively inconsistent. This inconsistency makes downstream debugging significantly harder, because the root cause of a discrepancy may lie not in the function being tested but in a shared idiom applied differently across functions. Developers for statistical packages often care more about the consistency of the codebase since elegance has a great impact on the readability and maintainability of the underlying rigorous mathematics. To make the converted package easier for future maintainers to understand and extend, it is crucial to ensure that the same underlying issue is handled in the same way across all functions.

We address these limitations with a seven-phase methodology that decomposes the translation problem into specialised, ordered steps, each executed by dedicated AI skills that orchestrate one or more sub-agents. The methodology generates a machine-readable catalogue of all language dependencies and their translation nuances *before* any code is written, ensuring consistency across all conversion agents. Functions are converted in topological dependency order, so each conversion agent can inspect the Python API of already-converted callees. The converted package is validated against the live R reference implementation using `rpy2` at a specified numerical tolerance, and an iterative automated bug-resolution loop continues until all tests pass. The resulting framework is designed to be reproducible, auditable, and extensible to packages of greater complexity.

We demonstrate the methodology on `KernSmooth` [6], a recommended R package implementing kernel density estimation, plug-in bandwidth selection, and local polynomial regression. `KernSmooth` serves as an ideal proof-of-concept: it has zero runtime dependencies on contributed CRAN packages (eliminating the recursive dependency problem), its computationally intensive work is fully delegated to compiled Fortran 77 routines (testing the methodology's ability to handle the R Fortran foreign-function interface), and its seven public functions implement well-defined algorithms with exact numerical specifications, enabling rigorous validation.

This project employs **Claude Code** (Anthropic) as the AI-assisted development environment, as it is the industry standard for vibe coding. The framework is described

at a level of generality intended for application beyond this specific example. All skills and agents are implemented as structured natural-language specifications stored in the `.claude/` directory, making them transparent, auditable, and reusable for future projects.

2 Results

2.1 Overview of the Methodology

[Figure 1 placeholder]

Seven-phase pipeline overview. A horizontal flowchart with seven labelled boxes (Phase 1 through Phase 7) connected by rightward arrows. Each box names the phase, lists the primary skill invoked in that phase, and indicates the dispatch mode of its sub-agents (sequential, parallel, or iterative). Input and output artefacts are shown as labelled arrows entering and leaving each box: structural analysis JSON (Phase 1 output / Phase 3 input), language dependency CSV and 46 conversion guides (Phase 3 output / Phase 4 input), converted Python module (Phase 4 output / Phases 5–7 input), and test files (Phase 6–7 output). Phase 2, which involves no skill invocation, is marked "Manual design." A legend distinguishes skill boxes (light blue) from sub-agent icons (gold) and manual steps (grey).

Figure 1: **Overview of the seven-phase AI-assisted R-to-Python conversion methodology.** The pipeline proceeds from static structural analysis (Phase 1) through build infrastructure (Phase 2), language dependency cataloguing (Phase 3), automated function conversion and assembly (Phase 4), code quality auditing (Phase 5), R test porting (Phase 6), and comprehensive test-suite construction with iterative bug resolution (Phase 7). The phase outputs form a directed chain of artefacts consumed by subsequent phases. Skills (top-level slash commands) and their sub-agents are indicated for each phase; the dispatch mode (sequential, parallel, or iterative) of each sub-agent fleet is shown explicitly.

The methodology is structured as a pipeline of seven phases (Fig. 1). Each phase is motivated by a distinct risk of error or inconsistency in the translation process; the phase is resolved by invoking one or more *skills*—top-level slash commands encoded as structured natural-language specifications stored under `.claude/commands/`—which orchestrate one or more *sub-agents*—specialised worker agents with independent LLM contexts whose specifications reside under `.claude/agents/` (see Supplementary Note 1 for the architecture of skills and agents; full specifications are given in Supplementary Note 2 and Supplementary Note 3). At the conclusion of each phase, the `/summarize-research-report` skill produced a structured phase summary saved to `docs/daily_summaries/`, providing a durable record of decisions made.

2.2 Phase 1: Static Structural Analysis

Before any translation work can begin, a precise machine-readable map of the source package's internal structure is required. Without it, the conversion agent for each R function must infer architectural facts independently, risking inconsistent dependency classification and missed foreign-function interface calls. The analysis results can also be used to generate human-readable dependency graphs, giving developers a clear picture of the package's architecture to keep track of. We therefore begin by analysing the source package statically and find all function dependencies and foreign-function interface calls for each function.

[Figure 2 placeholder]

Skill-agent orchestration architecture. A two-level diagram showing the general pattern. At the top level, a "Skill" box represents a slash command that reads multiple input files in parallel (shown as incoming arrows from file icons), applies context-construction logic, and then dispatches sub-agent instances. Two dispatch modes are illustrated side by side: (left) "Sequential dispatch" with a single agent instance processing items one at a time (topological order; used in Phases 1, 4, 6, 7a), and (right) "Parallel dispatch" with multiple simultaneous agent instances (used in Phase 3b). Each agent instance receives a per-item context bundle (R source fragment, JSON dependency map, conversion guide folder) and writes one output artefact. An "Iterative dispatch" variant (bottom right) shows the loop used in Phase 7b: run tests, dispatch fix-agent on first failure, reinstall package, repeat.

Figure 2: **Skill-agent orchestration architecture.** A skill (slash command) acts as an orchestrator: it assembles per-item context bundles from shared inputs and dispatches sub-agents in one of three modes. Sequential dispatch (Phases 1, 4, 6, and 7a) is used when each agent invocation must have access to the outputs of prior invocations. Parallel dispatch (Phase 3b) is used when agent invocations are independent and latency is the binding constraint. Iterative dispatch (Phase 7b) repeatedly invokes a fix-agent until a stopping criterion (all tests pass) is met.

A direct idea is to simply prompt the LLM to read all R source files at one time and generate a mapping of function dependencies and foreign-function interface calls of all functions. However, an R package may contain many separate files, each with multiple functions and dozens of call sites, and the LLM's context window is too small to process the entire source at once. To address this, the `/analyze-r-folder-dependencies` skill was designed to orchestrate a fleet of `@analyze-r-file-dependencies` sub-agents in a sequential dispatch pattern (Fig. 2). The `/analyze-r-folder-dependencies` skill (Supplementary Note 3) was invoked with the R source directory as its argument. The skill scans the target folder recursively for `.R` files and dispatches the `@analyze-r-file-dependencies` sub-agent (Supplementary Note 2) once per file, sequentially.

Each agent processes one R file at a time. It first extracts all function definitions,

```
{
    "function1": {
        "language_dependencies": ["dbeta", "dnorm"],
        "internal_dependencies": ["function2"],
        "external_dependencies": [".Fortran(F_blkest)"]
    },
    "function2": {
        "language_dependencies": ["fft"],
        "internal_dependencies": [],
        "external_dependencies": []
    }
}
```

Figure 3: Example of the JSON artefact produced by the `@analyze-r-file-dependencies` agent for a single R source file. The JSON object is keyed by function name, and each function maps to an object containing three lists: `language_dependencies` (calls to R built-in or standard-library functions), `internal_dependencies` (calls to other functions defined within the same package), and `external_dependencies` (calls through the Fortran or C foreign-function interface). This structured format allows subsequent agents to easily query the dependencies of each function for informed translation decisions.

then identifies all function calls within each function body. Extra scans for other files are performed only when necessary to ensure a comprehensive understanding of the dependency relationships. All function calls identified will be categorized into three different categories, which are:

- *Language dependencies*: calls to R built-in or standard-library functions, e.g. `dbeta`, `dnorm`, `fft`, `quantile`, `var`.
- *Internal dependencies*: calls to other functions defined within the same R file or other files in the same package, e.g. `linbin` calls `linbin2D`.
- *External dependencies*: calls through the Fortran or C foreign-function interface (`.Fortran()`, `.C()`, `.Call()`, or `.External()`), e.g. `.Fortran(F_blkest)`.

The agent produces a JSON artefact that maps each function name to its three types of dependencies. All functions in the same R source file will be included in a single JSON file, which contains a JSON object keyed by function name. An example of the JSON files is shown in Fig. 3. The JSON files will then be consumed by subsequent agents for language dependency extraction (Phase 3) and function conversion (Phase 4).

Because `KernSmooth` consolidates all R code into a single file (`KernSmooth/R/all.R`), exactly one agent invocation was required, and only one JSON file was generated. The result documented the dependencies of all 16 functions and detected eight distinct Fortran entry points.

With the structural analysis complete, a human-readable dependency graph was generated based on the JSON files. The graph serves as a visual reference for developers to understand the package's architecture and track the conversion progress. Additionally, to build a machine-readable directed acyclic graph (DAG) for indication of conversion

function	level	parents	children
.onAttach	0 (leaf)		
.onUnload	0 (leaf)		
bkde	0	linbin	
bkde2D	0	linbin2D	
dpih	0	bkfe; linbin	
dpik	0	bkfe; linbin	
dpill	0	blkest; cpblock; locpoly; rlbin; sdiag; sstdiag	
bkfe	1	dpih; dpik	linbin
blkest	1 (leaf)	dpill	
cpblock	1 (leaf)	dpill	
linbin2D	1 (leaf)	bkde2D	
locpoly	1	dpill	linbin; rlbin
sdiag	1	dpill	linbin
sstdiag	1	dpill	linbin
linbin	2 (leaf)	bkde; bkfe; dpih; dpik; locpoly; sdiag; sstdiag	
rlbin	2 (leaf)	dpill; locpoly	

Table 1: Dependency-level table for the 16 functions in `KernSmooth`. The "level" column indicates the function's depth in the dependency graph, with 0 representing root functions that are not called by any other function, and higher levels indicating deeper dependencies, with "(leaf)" tags for leaf functions. The "parents" column lists the immediate callers of each function, while the "children" column lists the immediate callees.

order, the JSON files were processed, and a table (1) was generated as a CSV file. The table classifies each function into a dependency level based on the longest path from any root function (Level 0, with no internal callers calling them) to the function in question, with leaf functions (those that call no other functions) at the lowest levels. The table also lists each function's immediate parents (callers) and children (callees), providing a clear map of internal dependencies that will guide the topologically ordered conversion in Phase 4. It reveals a three-level hierarchy of function dependencies, with five leaf functions (`blkest`, `cpblock`, `linbin2D`, `linbin`, and `rlbin`) that call no other R functions, four intermediate functions that call only leaf functions, and five root functions that call both leaf and intermediate functions.

2.3 Phase 2: Python Build Infrastructure

The conversion strategy retains the original Fortran 77 source files verbatim and compiles them into a Python C-extension via NumPy's `f2py` tool [15], isolating the risk of numerical errors and other bugs to the R-wrapper layer to be translated. Phase 2 established the packaging infrastructure necessary for both local development and broad distribution before any wrapper code was written.

This phase does not involve any skill invocation; the build system should be designed manually with the aid of Claude Code based on the architectural requirements identified in Phase 1. The `meson-python` backend was selected as the build system because it natively supports `f2py` targets and produces PEP 517-compliant wheels; details of the build system design, BLAS discovery logic, and PyPI distribution configuration are described in the Methods and Supplementary Note 5. The key design principle is that any numerical or logical differences between the Python and R packages can originate only in the R-to-Python wrapper translation, not in the underlying Fortran routines, providing a strong correctness guarantee.

2.4 Phase 3: Language Dependency Catalogue and Translation Guide Generation

While the internal and external dependencies of a function can be identified and understood easily since they are explicitly defined in the package source, the central risk of any LLM-assisted code translation is the inconsistent handling of language-specific semantic features across function conversion steps. This risk is especially acute for mathematical and statistical packages: an LLM operating on a single function may correctly handle the FFT normalisation convention in one instance but fail to do so in another, producing a package where the same underlying discrepancy is treated differently in different functions.

One may argue that these discrepancies are well-known and can be handled on a case-by-case basis during the conversion process in Phase 4. However, due to the random nature of the LLM, we cannot guarantee that all language dependencies will be correctly identified and translated whenever a function is converted. If there is not a unified, machine-readable reference of all language dependencies and their translation nuances, then the risk of silent numerical errors or unexpected bugs in the final Python package is unacceptably high. (Even if the translation is mostly correct, we cannot ensure the

consistency of choices or styles, which can make debugging significantly difficult.) Also, there are many language dependencies in R that have so many variations in their usage that it is hard to guarantee a correct translation without a comprehensive reference. For example, the `as.double()` function is used in many different contexts (such as on scalars, vectors, matrices, or even data frames), and its translation to Python can vary depending on the context. If we rely directly on the LLM to identify and translate each usage correctly during the conversion process, we may miss some cases or introduce errors. By contrast, if we systematically extract all language dependencies and their call sites in this phase, and then generate detailed conversion guides for each scenario for each dependency, we can ensure that the translation in Phase 4 is consistent and accurate across all functions, producing elegant and clean implementations for readability and maintainability of the mathematical logics.

As a consequence, Phase 3 is to systematically catalogue every R standard-library call in the source package and generate a dedicated, machine-readable translation guide for each distinct dependency before any conversion begins.

[Figure 3 placeholder]

Phase 3 pipeline. A two-stage diagram. Stage 1 (left): the `/extract-r-folder-language-dependencies` *skill dispatches the* `@extract-r-file-language-dependencies` *agent sequentially per R file, producing a 4-column CSV (language_dependency, function_name, line_number, call_body) with 370 rows. A utility script merges per-file CSVs into a combined table and sorts rows by dependency name, making entries for the same dependency contiguous. Stage 2 (right): the* `/generate-language-dependency-conversion-guides` *skill reads the combined CSV, groups rows by dependency name to find 46 unique dependencies, and dispatches 46 instances of the* `@generate-language-dependency-conversion-guide` *agent simultaneously in a single parallel batch. Each agent reads its CSV subset and the corresponding R source lines, optionally consults online R documentation, and writes a Markdown conversion guide to* `language_dependency_analysis/conversion_guides/`*. Arrows indicate data flow. An annotation highlights the key design choice: "Parallel dispatch; agents are context-independent."*

Figure 4: **Phase 3: language dependency extraction and parallel conversion guide generation.** The first stage extracts the complete call-site table (370 rows, 4 columns) from the R source via sequential agent dispatch. The second stage launches 46 sub-agent instances simultaneously—one per unique language dependency—each producing a Markdown guide specifying the exact Python translation strategy with code examples. Parallel dispatch is appropriate because the guide for each dependency is context-independent of all other guides.

Two skills are deployed in this phase (Fig. 4). Both of them follow the skill-agent orchestration architecture (Fig. 2), just like Phase 1. Firstly, the `/extract-r-folder-language-dependencie`

language_dependency	function_name	line_number	call_body
Re	bkde	78	Re(fft(kappa*gcounts, TRUE))
Re	bkde2D	156	Re(fft(rp*sp, inverse = TRUE)/(P1*P2))
Re	bkfe	232	Re(fft(kappam*Gcounts, TRUE))
abs	dpill	531	abs(th24Q)
any	locpoly	610	any(bandwidth <= 0)
as.double	blkest	265	as.double(x)
as.double	blkest	265	as.double(y)
as.double	blkest	267	as.double(xj)
as.double	blkest	267	as.double(yj)
as.double	blkest	267	as.double(coef)

Table 2: Excerpt from the language dependency call-site table. The full table contains 509 rows covering all 16 functions in `KernSmooth`. Each row corresponds to a single call site of an R standard-library function, with columns for the dependency name, enclosing function name, line number, and complete call expression. The table is sorted by dependency name, function name, and line number so that all call sites for each unique dependency are contiguous, and all call sites for each unique function are contiguous, facilitating the subsequent guide generation step.

skill (Supplementary Note 3) is invoked with the R source directory and the directory of Phase 1 structural analysis JSON files as arguments. The skill scans the source directory recursively to find all the `.R` files, and then dispatches the `@extract-r-file-language-dependencies` sub-agent (Supplementary Note 2) once per R file together with its structural analysis JSON, which pre-classifies each function's language dependencies, eliminating the need for the agent to infer dependency types from scratch for a second time. The agent rescans the R source line by line and records, for every individual call site, the dependency name, the enclosing function name, the line number, and the complete call expression (call body, including multi-line calls). A utility script merges per-file CSVs into a combined table sorted by dependency name, making all call sites for each dependency contiguous for subsequent processing. The result is a 509-row, 4-column CSV covering all 16 functions in `KernSmooth`, whose first few lines are shown in Table 2.

Secondly, the `/generate-language-dependency-conversion-guides` skill (Supplementary Note 3) was invoked with the R source directory and the combined CSV (Table 2) as arguments. The skill identified 46 unique dependency names in the CSV and launched 46 instances of the `@generate-language-dependency-conversion-guide` sub-agent (Supplementary Note 2) in a *single parallel batch*. Parallel dispatch is appropriate here because the translation guide for each dependency is logically independent of all other guides; there is no ordering constraint, and parallelism reduces wall-clock time from $O(N)$ to $O(1)$ agent invocations. Each agent received its assigned dependency's CSV subset, read the corresponding source lines in `KernSmooth` for contextual understanding, and optionally consulted the R online documentation for complex or non-obvious behaviors. The agent then produced a structured Markdown guide specifying the correct Python translation strategy with concrete code examples for each scenario, where functions with different types of calling arguments are considered

different scenarios, and separate examples are provided. To generate clean and useful guides, the agents can refer to Python online documentation. Each Markdown guide includes four parts:

1. Overview: a concise summary of the dependency's functionality and its translation challenges.
2. Contextual Usage Analysis: a detailed examination of all call sites in the source package, categorizing them by usage patterns and argument types.
3. Python Conversion Strategy: a comprehensive overview of the approach to translating the dependency's functionality into Python, including any necessary adaptations or workarounds.
4. Step-by-step Conversion Examples: for each unique identified usage pattern, a concrete example of the R code and its corresponding Python translation, with explanations of any non-obvious translation decisions.

The 46 resulting conversion guides were saved to a directory, which works like a catalog for the conversion agents in Phase 4 to look up. The most consequential guides—those where an incorrect translation would produce a silent numerical error with no runtime exception—are described in the Methods. Although no manual correction of the guides was performed, their structured format makes targeted human review and correction straightforward, providing a control point that constrains the risk of errors propagating into the following phases. This is a key advantage over an unguided single-pass LLM translation: the guides make every language-semantic decision explicit, auditable, and consistent.

2.5 Phase 4: Automated Function Conversion and Package Assembly

With the dependency map (Phase 1), build infrastructure (Phase 2), and translation guides (Phase 3) in place, Phase 4 executes the actual language translation from R to Python.

To determine the basic translation block is not a mindless task. Since most typical R packages use functional programming paradigms (although there is OOP in R, they are less prevalent in practice), there are two choices for the basic translation block: functions or entire files. During Phase 1 and Phase 3, the independent sub-agents always process entire R files at once, since they are the basic units of input and output (IO) stream, and the contexts are relatively simple. Now, to convert the entire R package into Python, the agents should not only take in the original R code, but also the dependency maps (the JSONs and CSV from Phase 1), generated conversion guides in Markdown (from Phase 3), the R and Python online language documentation, and the Python code that was already generated as a reference in case the future code needs to call them. Such a large number of contexts makes the translation agents heavily loaded, making it hard to achieve optimal performance when processing a whole file. Splitting the R files into separate R functions and translating them one by one can easily release the heavy lifts of the agents, forcing them to focus on the translation task without much irrelevant information. As a result, functions are chosen as the basic translation block for the agents.

Two challenges make the conversion step non-trivial despite all the preparation made in previous phases. First, functions must be converted in dependency order: a callee's Python API must exist (and be fixed) before the caller's conversion agent can write correct call expressions, which means children should be converted before their parents (see Table 1). Second, the Fortran foreign-function interface in Python (via `f2py`) differs fundamentally from R's `.Fortran()` in how it handles output arguments (see Methods), and this difference is not visible from the R source, requiring targeted awareness in conversion agents.

[Figure 4 placeholder]

Phase 4 conversion pipeline. Left panel: a dependency level diagram showing the three-level hierarchy of the 16 KernSmooth functions (Level 2: linbin, rlbin; Level 1: bkfe, blkest, cpblock, linbin2D, locpoly, sdiag, sstdiag; Level 0: .onAttach, .onUnload, bkde, bkde2D, dpih, dpik, dpill). Arrows show internal calls. A bottom-to-top arrow labelled "Conversion order" indicates that the `/convert-r-file-to-python` *skill processes Level 2 first, then Level 1, then Level 0, sequentially. Right panel: the per-function agent invocation. A single* `@convert-r-function-to-python` *agent instance receives four inputs (R source fragment, JSON dependency map, 46 conversion guides, output folder) and writes one JSON artefact. The sequential arrow between function boxes in the left panel is annotated: "Sequential: each agent reads prior outputs of internal callees." The final assembly step (skill:* `/combine-python-functions-into-file`*) merges 16 JSON artefacts into* `__init__.py`*.*

Figure 5: **Phase 4: topologically ordered function conversion and package assembly.** Functions are converted in leaf-to-root order so that each conversion agent can inspect the Python calling convention of already-converted internal dependencies. The `/convert-r-file-to-python` skill derives the conversion order from the dependency-level CSV produced in Phase 1 and dispatches the `@convert-r-function-to-python` sub-agent sequentially. The `/combine-python-functions-into-file` skill then assembles the 16 resulting JSON artefacts into the final `__init__.py`, correcting import paths and deduplicating imports.

The `/convert-r-file-to-python` skill (Supplementary Note 3) is invoked with the R source file, with the structural analysis JSONs (Fig. 3 from Phase 1), the dependency-level CSV (Table 1 from Phase 1), the conversion guide folder (from Phase 3), and an output folder as arguments (Fig. 5). The skill parses the dependency-level CSV to derive a topological ordering of the 16 functions, from deepest leaves (Level 2) to roots (Level 0), and dispatches the `@convert-r-function-to-python` sub-agent (Supplementary Note 2) *sequentially* once per function in that order. Sequential dispatch here is not a performance limitation but a correctness requirement: each agent reads the converted artefacts of the function's internal callees to understand their Python signatures and calling conventions, and those artefacts must already exist when the agent is invoked. Each agent receives the R source fragment for the dispatched function,

```
{
  "imports": [
    "import numpy as np",
    "from KernSmooth import _KernSmooth"
  ],
  "function_prototype": "def rlbin(X: np.ndarray[np.float64], Y: np.ndarray[np.float
  "function_body": [
    "    n = len(X)",
    "    M = len(gpoints)",
    "    trun = np.int32(1) if truncate else np.int32(0)",
    "    a = np.float64(gpoints[0])",
    "    b = np.float64(gpoints[-1])",
    "    xcnts = np.zeros(M, dtype=np.float64)",
    "    ycnts = np.zeros(M, dtype=np.float64)",
    "    _KernSmooth.rlbin(np.asarray(X, dtype=np.float64), np.asarray(Y, dtype=np.f
    "    return {\"xcounts\": xcnts, \"ycounts\": ycnts}"
  ]
}
```

Figure 6: Example of the JSON artefact produced by the `@convert-r-function-to-python` agent for a single R function (`rlbin`). The JSON object contains three fields: `imports` (a list of import statements required by the function), `function_prototype` (the complete Python function signature with type annotations), and `function_body` (a list of strings representing the lines of the function body). This structured format allows for systematic assembly into a Python file while ensuring that all necessary imports and correct function signatures are included.

the function's JSON dependency map extracted from the structural JSON, the folder of language dependency conversion guides, and the output folder for the conversion results. Unambiguous type annotations are requested to be explicit for the conversion to be semantically and logically accurate. The conversion results are not saved as Python files for now. Instead, they are self-contained JSON encodings of the converted Python functions, including imports, function prototypes, and function bodies, one of which is shown in Fig. 6. This format makes it easy for the agents that are converting callers to extract the necessary information (such as function prototypes, types of parameters, and types of return values) of the callees, which are already converted. The 16 resulting JSON artefacts named after the functions for `KernSmooth` were written to the output folder.

Following conversion, the `/combine-python-functions-into-file` skill (Supplementary Note 3) assembles the JSON artefacts into final Python files. Before writing, the skill scanns the `meson.build` and the existing module skeleton to verify the correct relative import path for the compiled Fortran extension, then corrected three categories of systematic errors present in the per-function JSON artefacts: absolute import paths were replaced with relative imports (e.g., `from . import _KernSmooth`); redundant cross-function imports were removed (functions sharing a module need not import each other); and duplicate third-party import statements were merged. The final import block was sorted in PEP 8 standard-library → third-party → local order. Syntax correctness is confirmed immediately by `ast.parse()`. After Phase 4, the package `r2py_kernsmooth` is in a runnable state with all 16 functions converted.

2.6 Phase 5: Code Quality Audit and Namespace Refinement

The automated conversion pipeline prioritises functional correctness. Phase 5 was a systematic audit and refactoring pass addressing three categories of concerns not directly handled by the conversion agents: type annotation completeness, namespace hygiene, and residual code-quality defects from the porting process.

This phase does not invoke any new skill; it is a manual review with the aid of Claude Code upon the assembled `__init__.py` guided by comparisons against the R source. The audit identifies and resolves: (i) bare container-type annotations lacking element type parameters (corrected to fully parameterised forms, e.g., `np.ndarray` should be `np.ndarray[Any, np.dtype[np.float64]]`); (ii) a silent crash bug in three public functions where a `None`-typed optional parameter was unconditionally passed to `np.asarray()`, producing a degenerate array that caused a `TypeError` at the subsequent `len()` call; (iii) a 27-line bandwidth-discretization block duplicated identically across three functions, extracted into a private helper `_discretize_bandwidth()`; (iv) an 8-line partial-string-match block duplicated across four call sites, extracted into `_resolve_choice()`; and (v) dead code consisting of R lifecycle hooks (`.onAttach` and `.onUnload` transliterations) that are unreachable in Python's import machinery. Every error and warning message in the Python module is also verified against the corresponding `stop()` and `warning()` calls in the R source, and three discrepancies in message format or conditions were corrected. Details of the specific corrections are given in the Methods.

2.7 Phase 6: R Test Conversion and Validation Infrastructure

In general, testing the converted Python package against the original R implementation is much easier than testing a brand-new software project, because the R source provides a complete executable specification of intended behavior. Especially for Claude Code, that is a big advantage, since it can focus mainly on the input parameters of each function, with little consideration needed for the output (return values or exceptions), as we have the original R output as the reference.

A validated test suite grounded in the R reference implementation is the primary mechanism for detecting semantic errors in the converted package `r2py_kernsmooth`. Phase 6 ports the two regression unit tests shipped with the original R `KernSmooth` package into Python equivalents and upgrades them to assertion-based numerical comparisons using `rpy2` [1], which calls the R runtime at test time from Python.

The `/convert-r-tests-to-python` skill (Supplementary Note 3) is invoked with the folder of R tests, the R package directory, the Python package directory, and an output folder for the converted tests as arguments. The skill scans the R test folder recursively for `.R` scripts and dispatches the `@convert-r-test-to-python` sub-agent (Supplementary Note 2) once per file, sequentially, producing a Python script for each. The agent parses the R test script, identifies all calls to the R functions under test, and generates a Python test script that uses `rpy2` to call the original R functions with the same inputs and compares the results against the converted Python functions with assertions from `pytest`, which is a standard testing framework for Python.

The resulting tests use `rpy2` throughout: R packages are loaded once at module level with `importr()`; input data is passed as `ro.FloatVector` or `ro.IntVector`; results are extracted with `.rx2()` and converted to NumPy arrays; and numerical agreement is asserted with `numpy.testing.assert_allclose` at explicitly specified tolerances (`rtol=1e-10` for grid coordinates, `rtol=1e-6` for density and regression estimates). This `rpy2`-based testing pattern—rather than hardcoded reference values—was adopted deliberately: hardcoded values require manual regeneration when the R reference changes and may introduce spurious failures due to platform-specific floating-point differences, whereas `rpy2`-based tests always compare against the current R implementation. All four tests passed after some specific bug fixes, where the details are described in Methods and Supplementary.

2.8 Phase 7: Comprehensive Test Suite Construction and Bug Resolution

At the conclusion of Phase 6, the test suite comprised four tests covering only three of the seven public functions, which are far from enough for the full functionality of the package. Phase 7 systematically builds a suite of comprehensive positive, negative, and edge-case covered unit tests for all seven public functions and employs an iterative automated bug-resolution loop to bring the full suite to zero failures.

The three different unit test categories are defined as follows:

- Positive tests: cases where the input parameters are valid, and the function is expected to return a correct result. These tests need to cover all documented

[Figure 5 placeholder]

Test suite statistics. Two panels. Left: a stacked horizontal bar chart with one bar per public function (`bkde`, `bkde2D`, `bkfe`, `dpih`, `dpik`, `dpill`, `locpoly`). Each bar is divided into three segments: positive tests (blue), negative tests (orange), and edge-case tests (green). The total count is annotated at the right end of each bar. Values from Table 1 (`bkde`: 57, `bkde2D`: 63, `bkfe`: 49, `dpih`: 71, `dpik`: 119, `dpill`: 83, `locpoly`: 74; total 516 generated + 2 from Phase 6 = 518). Right: a two-column comparison showing "Initial pass rate" vs. "Final pass rate" as horizontal bar segments (or two bars per function). Five functions initially pass all tests; only `dpill` has 62 initial failures, all resolved after three rounds of bug fixing (the iterative `/fix-bugs-found-in-tests` loop). Final pass rate: 518/518.

Figure 7: **Test suite coverage and resolution of initial failures.** 516 tests were generated by the `/generate-python-file-tests` skill (7 public functions × 3 categories: positive, negative, edge-case), supplemented by 4 tests ported in Phase 6, for a total of 518. All but 62 `dpill` tests passed on first run. Three distinct bugs were identified by the `/fix-bugs-found-in-tests` iterative loop and resolved in three sub-agent invocations; all 518 tests pass after resolution.

parameter combinations and typical use cases that are relevant to the function's intended behavior.

- Negative tests: cases where the input parameters are invalid or where the function is expected to raise an error. These tests cover all explicit error conditions documented in the R source, such as invalid parameter values, mismatched input lengths, or other conditions that should trigger exceptions.
- Boundary and edge-case tests: cases that test the function's behavior at the boundaries of valid input or with special values. These tests cover boundary inputs such as empty arrays, single-element arrays, `NaN`/`Inf` values, and degenerate grid ranges, which may reveal issues with numerical stability, handling of special cases, or other edge conditions that are not covered by typical positive or negative tests.

This comprehensive coverage strategy is designed to maximize confidence in the numerical correctness and robustness of error handling of the converted package. Since it is a statistical package focusing on mathematical logic, more complex tests of software engineering are not required.

The `/generate-python-file-tests` skill (Supplementary Note 3) is invoked with the Python module as its argument. The skill parses `__init__.py` to enumerate all functions and identifies the seven user-facing public functions by consulting the KernSmooth CRAN reference manual; the remaining internal utilities with no R documentation were excluded. The `@generate-python-function-tests` sub-agent (Supplementary Note 2) is then dispatched *sequentially* once per public function. Sequential dispatch is preferable here to avoid redundant `rpy2` initialisation overhead and to allow later agents to avoid replicating edge cases already covered by earlier ones. Each agent parses

the function signature and docstring, fetched the corresponding R documentation page to grasp a deep understanding of the interrelationships between all parameters and return values, and generated three test files: positive cases (`test_fn_positive.py`), negative cases (`test_fn_negative.py`), and edge cases (`test_fn_edge.py`). All tests use `rpy2` to obtain and catch live R reference values and exceptions.

For tests that are expected to throw exceptions (all negative tests and some edge cases), the agent applies a principled strategy for cases where R and Python raise errors with different messages or where one language raises while the other returns a non-finite result (most of these cases are due to differences between language features, such as lazy evaluation vs. eager evaluation): such cases are documented by a `UserWarning` rather than treated as test failures, providing an honest record of behavioural divergences. For example, if both R and Python throw an error, the test should be considered passed. If R throws an error but Python does not, the test should be considered failed, and the agent will automatically investigate the discrepancy and determine whether the Python implementation needs to be updated to match R's behavior or if the test case is not applicable due to language differences. There are some cases where we accept the tests where one language throws an error, while the other does not, but returns arrays with `NaN` or `Inf` values. Under those circumstances, the tests are written in a way determined by the agent itself to be informative about the language differences and the expected behavior in each language. The agents will automatically run the tests and see whether the behaviors match the expectations. When they find failures because the tests are poorly designed, they are going to fix them until success. 516 tests were generated across the seven functions (Table 3; Fig. 7); 454 passed on the first run, with 62 failures concentrated entirely in `dpill`.

Table 3: **Test suite composition and initial pass rates.** Tests were generated by the `/generate-python-file-tests` skill and cover all seven public functions of `r2py_kernsmooth`. Positive tests cover all documented parameter combinations; negative tests cover all explicit error conditions; edge tests cover boundary inputs (empty arrays, single elements, NaN/Inf values, degenerate grid ranges). All failures were in `dpill` and were resolved by the `/fix-bugs-found-in-tests` iterative loop.

Function	Positive	Negative	Edge	Total	Initial pass
`bkde`	28	9	20	57	57
`bkde2D`	28	12	23	63	63
`bkfe`	24	9	16	49	49
`dpih`	32	15	24	71	71
`dpik`	52	23	44	119	119
`dpill`	44	17	22	83	21
`locpoly`	38	11	23	74	74
Total	**246**	**96**	**172**	**516**	**454**

Then we need to fix the bugs detected. The `/fix-bugs-found-in-tests` skill (Supplementary Note 3) is invoked to resolve the 62 `dpill` failures. The skill follows an iterative protocol: it runs the full test suite with `pytest`, isolates the first failing test, dispatches the `@fix-bug-found-in-test` sub-agent (Supplementary Note 2) with the

failing test's traceback and the relevant source context, reinstalls the package if the fix modifies library code, and repeats until all tests pass. This iterative dispatch mode handles bugs that only become visible after prior bugs are fixed, which is precisely the case here: the three bugs in `dpill` formed a cascade where each prior bug masked the subsequent one (see Methods for details). Three sub-agent invocations are sufficient to resolve all failures. When resolving identified bugs, the skill and agents ensure that the package is rebuilt and all tests are re-run to confirm the fixes. The agents are clever enough to delve deep into the intermediate variables automatically passing through in the calling stacks. After the final package rebuild, running the complete suite with `python -m pytest r2py_kernsmooth/tests/ -q` produced 518 passed, 0 failed in 6.40 seconds under Python 3.14.4 / pytest 9.0.3. All seven public functions achieve numerical agreement with R to 6–10 significant figures across all positive and edge test cases.

3 Discussion

The methodology presented here demonstrates that AI-assisted translation of R statistical packages to Python can produce a numerically correct, well-tested, distributable package with broad cross-platform binary wheel coverage. The 518-test result establishes numerical agreement with the R reference at tolerances (`rtol` $= 10^{-6}$ to 10^{-10}) well within the numerical precision relevant to any statistical application. The framework is not specific to `KernSmooth`: the seven phases, the skill-agent architecture (Fig. 2), and the rpy2-based validation strategy are applicable to any R package whose source structure is accessible. Several limitations of the current implementation, however, bound its immediate generalisability and motivate future extensions.

External dependency footprint. `KernSmooth` has zero runtime dependencies on contributed CRAN packages, relying solely on five functions from R's built-in `stats` package. This minimal footprint means that the methodology, as applied here, needed to handle only the translation of R standard-library idioms—a finite and cataloguable set. Packages with substantial external CRAN dependencies present a more complex challenge. A package such as `glmnet` depends on `Matrix` (sparse linear algebra) and `foreach` (parallel iteration), each of which requires its own Python equivalent. The recursive application of the methodology—applying Phase 1 through Phase 7 to each CRAN dependency, bottom-up—is conceptually straightforward but operationally demanding: the search space of dependency-specific translation nuances grows, and the interaction between dependency translations must be managed carefully. The current framework does not maintain a cross-project persistent knowledge base of validated language-dependency translation decisions; building such a database, seeded by the conversion guides generated in each project, is a natural next step towards handling the long tail of package dependencies efficiently.

Fortran and C foreign-function interface convention. `KernSmooth` calls its Fortran routines exclusively through R's `.Fortran()` and `.C()` interfaces, which pass arguments as simple C-compatible arrays by reference. This interface is mechanically straightforward to bridge via f2py: both the R and Python sides agree on the binary representation of the data, and the only translation work required is mapping R's

named-list return convention to f2py's in-place output-argument convention. Many important R packages use the more complex `.Call()` interface instead. `.Call()` passes arguments as *SEXP* objects—R's native C data structures, which encode R-specific type information (length, attributes, protection bits) that is not present in a plain C array—and can return arbitrary R objects including lists, environments, and closures. Converting a package that uses `.Call()` requires either (i) reimplementing the C-level computation in Python or Cython, losing the correctness guarantee of the verbatim Fortran/C source, or (ii) writing a thin SEXP-compatible C shim that bridges the `.Call()` interface to a Python-callable equivalent. A package such as `rpart`, which implements recursive partitioning entirely in C via `.Call()`, would require the latter approach. Developing a general methodology for `.Call()`-based packages is a significant open problem; it may require tool support for mechanically lowering SEXP-typed C code to a Python-accessible representation.

LLM consistency and correctness. The methodology uses structured pre-processing (the Phase 3 conversion guides) and topological ordering (Phase 4 sequential dispatch) to reduce, but not eliminate, the risk of LLM inconsistency. The f2py return-value bug discovered at the end of Phase 4—where the conversion agent for `linbin` incorrectly subscripted the `None` return value of an f2py-wrapped subroutine, while the agent for `rlbin` correctly used in-place output semantics—demonstrates that independent agent invocations can diverge even when processing structurally similar code. The iterative Phase 7 bug resolution loop mitigates this risk at the test level, but it requires that the test suite exercise the buggy code paths. Silent numerical errors that do not manifest in any test case remain possible. Future work could strengthen the methodology by incorporating formal equivalence checking [13] for individual converted functions, using the R source as the reference specification.

Scalability and cost. The methodology as described involves 46 parallel agent invocations in Phase 3, 16 sequential invocations in Phase 4, 7 sequential invocations in Phase 7a, and a variable number of iterative invocations in Phase 7b. For `KernSmooth`, this is computationally modest. For a package with hundreds of functions and dozens of CRAN dependencies, the number of agent invocations grows proportionally, and the wall-clock time for Phase 4 (which is necessarily sequential) scales linearly with the function count. Strategies to mitigate this include batching multiple simple functions into a single agent invocation where their dependency relationships permit, and parallelising independent sub-trees of the dependency graph.

Generalisation and next steps. The immediate next extensions of the framework are to packages that use `.Call()` (testing the FFI translation capability) and to packages with external CRAN dependencies (testing the recursive application). An automated Phase 2 skill that generates the `meson.build` and `pyproject.toml` from a structural analysis of the source package's `src/` directory would reduce the sole remaining manual phase. Longer term, the Phase 3 conversion guides could seed a shared, cross-package translation knowledge base, making each successive project cheaper to translate as the coverage of validated language-dependency guides grows.

References

[1] Laurent Gautier. *rpy2: R in Python*. https://rpy2.github.io. Python package, version 3.5. 2024.

[2] Kevin Ushey, J. J. Allaire, and Yuan Tang. *reticulate: R interface to Python*. https://CRAN.R-project.org/package=reticulate. R package, version 1.34.0. 2023.

[3] Mark Chen et al. "Evaluating large language models trained on code". In: *arXiv preprint arXiv:2107.03374* (2021).

[4] Baptiste Rozière et al. "Unsupervised translation of programming languages". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 20601–20611.

[5] Yue Wang et al. "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation". In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 2021, pp. 8696–8708. DOI: 10.18653/v1/2021.emnlp-main.685.

[6] M. P. Wand and M. C. Jones. *Kernel Smoothing*. London: Chapman & Hall, 1995. ISBN: 9780412552700.

[7] S. J. Sheather and M. C. Jones. "A reliable data-based bandwidth selection method for kernel density estimation". In: *Journal of the Royal Statistical Society: Series B* 53.3 (1991), pp. 683–690. DOI: 10.1111/j.2517-6161.1991.tb01857.x.

[8] D. Ruppert, S. J. Sheather, and M. P. Wand. "An effective bandwidth selector for local least squares regression". In: *Journal of the American Statistical Association* 90.432 (1995), pp. 1257–1270. DOI: 10.1080/01621459.1995.10476630.

[9] M. P. Wand. "Fast computation of multivariate kernel estimators". In: *Journal of Computational and Graphical Statistics* 3.4 (1994), pp. 433–445. DOI: 10.1080/10618600.1994.10474652.

[10] Travis E. Oliphant. "Python for scientific computing". In: *Computing in Science & Engineering* 9.3 (2007), pp. 10–20. DOI: 10.1109/MCSE.2007.58.

[11] Charles R. Harris et al. "Array programming with NumPy". In: *Nature* 585 (2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2.

[12] Pauli Virtanen et al. "SciPy 1.0: fundamental algorithms for scientific computing in Python". In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[13] Sahil Bhatia et al. "Verified code transpilation with LLMs". In: *Advances in Neural Information Processing Systems*. Vol. 37. arXiv:2406.03003. 2024.

[14] Hanliang Zhang et al. "Scalable, validated code translation of entire projects using large language models". In: *arXiv preprint arXiv:2412.08035* (2024).

[15] Pearu Peterson. "F2PY: A tool for connecting Fortran and Python programs". In: *International Journal of Computational Science and Engineering* 4.4 (2009), pp. 296–305. DOI: 10.1504/IJCSE.2009.029165.

4 Methods

4.1 KernSmooth Package Architecture

KernSmooth [6] (v. 2.23-26, license: Unlimited) implements kernel density estimation (bkde, bkde2D), kernel functional estimation (bkfe), plug-in bandwidth selection (dpih, dpik, dpill), and local polynomial regression (locpoly). All R code resides in a single file, KernSmooth/R/all.R, defining 16 functions: 7 exported via NAMESPACE and 9 internal helpers. The internal call graph has three levels: Level 0 (7 entry points callable by the user), Level 1 (7 intermediate helpers called by Level 0 functions), and Level 2 (2 linear binning primitives called by Level 1). linbin (Level 2), which performs one-dimensional linear binning via the Fortran linbin subroutine, is the most central function, called by seven distinct callers. The computational pattern throughout the package is a two-phase strategy: (1) bin raw data onto a regular grid using linear binning (which preserves the first moment within each bin interval, unlike histogram binning), and (2) convolve the bin counts with a kernel weight vector, either via FFT or via Fortran local-polynomial routines. This reduces computational complexity from $O(n \cdot g)$ to $O(n + g \log g)$, where n is the sample size and g is the grid size [9].

The package's NAMESPACE file declares useDynLib(KernSmooth, .registration = TRUE, .fixes = which registers all Fortran entry points at package load time and applies the F_ prefix to their R-level names. The package's Fortran source comprises 11 fixed-form Fortran 77 files in KernSmooth/src/, implementing the eight subroutines called via .Fortran(): linbin, lbtwod, blkest, cp, locpol, rlbin, sdiag, and sstdg. Three LINPACK routines (dgedi, dgefa, dgesl) are included for LU decomposition. See Supplementary Note 5 for the complete file listing and dependency map; Supplementary Fig. 1 shows the full function dependency graph.

4.2 Build System Design

The Python package r2py_kernsmooth uses meson-python as its PEP 517 build backend, with the full build definition in r2py_kernsmooth/meson.build. The f2py [15] wrapper is generated by NumPy (v. 2.4.3) as a Meson custom target: python -m numpy.f2py ${source The -lower flag lowercases Fortran symbol names to match GFortran conventions. An initial build template incorrectly listed the generated wrapper file as _KernSmooth-f2pywrappers2.f90 (the Fortran 90 free-form variant); because all sources are Fortran 77 fixed-form, f2py generates _KernSmooth-f2pywrappers.f (the fixed-form variant), and the meson.build outputs list was corrected accordingly.

Two Fortran files were added from Netlib beyond the original KernSmooth sources: dqrdc.f and dqrsl.f (LINPACK QR decomposition, G. W. Stewart, 1978). These are required because blkest.f and cp.f call dqrdc_ and dqrsl_, respectively, which R satisfies from its own internal LINPACK copy; OpenBLAS does not provide LINPACK QR routines. BLAS discovery in meson.build follows a five-stage fallback chain (pkg-config, then find_library for openblaso, openblas, and blas, then a hardcoded HPC-node path) that ensures portability across Linux, macOS, and Windows while remaining functional on the development cluster. Details of the PyPI distribution configuration (cibuildwheel, GitHub Actions workflow, platform coverage) are given in Supplementary Note 5.

4.3 f2py Interface Semantics and Compatibility

The most structurally significant difference between the R and Python Fortran FFI layers is in the handling of output arguments. R's `.Fortran()` always returns a named list of all arguments, including arrays modified in-place by the Fortran subroutine; the R idiom `.Fortran(F_linbin, ...)$gcnts` extracts the modified output array from that list. An f2py-wrapped Fortran subroutine, by contrast, returns `None` unconditionally; output array arguments are modified in-place via C pointer, and the Python caller reads results by examining the pre-allocated arrays passed in. Scalar output arguments require special treatment: a Python `np.float64` scalar is immutable, so passing one does not allow Fortran to write back; a length-1 NumPy array (`np.zeros(1, dtype=np.float64)`) is mutable and passes a writable pointer. See Supplementary Fig. 2 for a schematic comparison of the two calling conventions.

A second compatibility issue specific to NumPy 2.4.3 is the absorption of Fortran dimension scalars. When f2py processes a subroutine with parameters such as `ss(M, ippp)`, it infers the dimensions `M` and `ippp` from the shapes of the passed arrays at call time, removing them from the required positional argument list and making them optional with defaults inferred from array shapes. The Phase 4 conversion agents were not aware of this behaviour and retained the dimension integers at their original Fortran-order positions, causing all subsequent arguments to be misaligned. For the most complex affected subroutine (`locpol`), this reduced the call from 19 to 16 positional arguments after the correction.

A third compatibility issue affected the `blkest` subroutine's scalar output arguments (`sigsqe`, `th22e`, `th24e`), which f2py declared as `input float` (passed by value) rather than as writeable output arguments, because they appear as plain Fortran `DOUBLE PRECISION` scalars rather than as array arguments. The fix required generating a full f2py signature file (`r2py_kernsmooth/src/_KernSmooth.pyf`) from the Fortran sources and adding explicit `intent(out)` annotations to the three scalars in the `blkest` block. The `meson.build` custom target was updated to pass the `.pyf` file as the sole f2py input, causing f2py to honour the explicit intent declarations. After this fix, `blkest` returns a Python tuple of three floats rather than receiving pre-allocated length-1 arrays.

4.4 Key Language Translation Decisions

Of the 46 language dependencies catalogued in Phase 3, eight required translation decisions that would produce silent numerical errors if handled naively. These are described here; further nuances are given in Supplementary Note 4.

FFT normalisation. R's `fft(x, inverse=TRUE)` returns the raw unnormalised inverse DFT (sum without division by N). NumPy's `np.fft.ifft` normalises by $1/N$ by default. The R source explicitly divides the inverse-FFT result by the grid size P (or $P_1 \times P_2$ for 2D); these divisions must be omitted in Python.

Integer width. R's Fortran `INTEGER` is 32-bit. Passing a `np.int64` array to a Fortran subroutine expecting 32-bit integers silently corrupts values on 64-bit systems. All integer arrays passed to Fortran were typed as `np.int32`.

Variance denominator. R's `var(x)` applies Bessel's correction ($n-1$ denominator). NumPy's `np.var` defaults to the population variance (n denominator). All occurrences of `sqrt(var(x))` in the R source were translated as `np.std(x, ddof=1)`.

Missing-argument detection. R's `missing(param)` returns `TRUE` if the caller did not supply the argument. The Python equivalent uses a `None` sentinel default (`param=None`) with an `if param is None:` guard, ensuring that the default-computation logic executes only when no value is provided.

Negative-base fractional exponentiation. Several plug-in bandwidth formulas involve odd roots of potentially negative quantities (e.g. $(-3\sqrt{2/\pi}/\hat{\psi}_6)^{1/7}$). R silently returns a real-valued result; Python's `**` operator raises `ValueError` for negative bases with non-integer exponents. The correct Python idiom is `math.copysign(abs(x)**(1/n), x)`.

Column-major memory order. R's `matrix()` and all Fortran arrays use column-major (Fortran) storage. NumPy defaults to row-major (C) order. All 2D arrays passed to or received from Fortran subroutines were created or reshaped with `order='F'`.

Partial string matching. R's `match.arg()` supports unambiguous prefix matching (e.g. `"ep"` matches `"epanechnikov"`). No Python standard-library equivalent exists. A private helper `_resolve_choice(val, choices)` performing exact-then-prefix matching was implemented and used at all four call sites.

R lazy default-argument evaluation. R evaluates default argument expressions at the time of first use inside the function body, not at call time. In `dpill`, the default `range.x = range(x)` is not evaluated until after `x` has been sorted and trimmed; the effective range is therefore the trimmed range. Python's eager evaluation of defaults requires explicitly placing the `range_x = (x[0], x[-1])` assignment in the function body *after* the trimming step. The Phase 4 conversion agent had placed it before, causing `dpill` to use the pre-trim range and produce bandwidth values that diverged from R for right-skewed inputs.

4.5 Test Validation Strategy

All 518 tests use rpy2 [1] to obtain reference values from the live R `KernSmooth` package (v. 2.23-26). R packages are loaded once at module level with `importr()`. Input vectors are passed as `ro.FloatVector` or `ro.IntVector`; named arguments are mapped using `rpy2`'s keyword convention. Results are extracted with `.rx2()` and converted to NumPy arrays via `np.array()`. Numerical agreement is asserted with `numpy.testing.assert_allclose` at `rtol=1e-10` for grid coordinates and `rtol=1e-6` for density and regression estimates, reflecting the precision of the double-precision Fortran computations. Behavioural divergences (cases where R and Python raise different exceptions, or where one returns `NaN`/`Inf` while the other raises) are documented by `UserWarning` rather than assertion failure. For the `locpoly` tests, the Prestige dataset is extracted directly from R via `ro.r("Prestige$income")` to guarantee exact row correspondence with the R reference. The complete per-function test content is described in Supplementary Note 7.

Declarations

Data and code availability

The Python package `r2py_kernsmooth` and all project artefacts (structural analysis JSONs, language dependency CSVs, conversion guides, converted Python module, test suite) are available at https://github.com/caiyufei8/python-KernSmooth. The Python package is additionally available as a standalone repository at https://github.com/caiyufei8/r2py_kernsmooth. The original R `KernSmooth` package (v. 2.23-26) is available from CRAN under an Unlimited license.

Author contributions

Y.C. designed the methodology, implemented all phases, wrote the code, and wrote the manuscript.

Competing interests

The author declares no competing interests.

Acknowledgements

The author thanks the HPC facility of [Institution] for providing computational resources. The AI-assisted development was performed using Claude Code (Anthropic).